VULNSCAN · INTERNALS HANDBOOK

How vulnscan works

A line-by-line explanation of every stage between a hostname and a client-ready report: how services are identified, how a finding earns its severity, where the CVSS number comes from, what the advisory database really stores, and which failures the design deliberately accepts. Written to be read with the code open beside it.

Version 0.1.0 **Rule IDs** 58 **Tests** 304 **Runtime** Python 3.10+, standard library **Updated** 6 November 2026

Every number, flag and file reference in this document was read out of the code in this repository, not from memory. Where the behaviour is subtle or the reasoning matters more than the mechanics, there is a **Why** note — those are the parts worth arguing about in a review.

Contents

01	What the tool is, and what it refuses to be
02	The pipeline, end to end
03	The data model: `PortService`, `Finding`, the report dict
04	Identity: ports locate, protocols decide
05	The two engines: nmap and native
06	Evidence: what is actually said on the wire
07	The rule catalogue (58 rules, grouped by what gates them)
08	Severity, confidence and CVSS
09	The offline advisory database
10	Guidance: ATT&CK, detection, next step
11	The report: JSON, HTML, CSV, PDF
12	The CLI surface and its exit codes
13	Testing: how 304 tests keep the rules honest
14	Recipes: extending the tool safely
15	Failure modes and known limits
16	Study plan and self-test
17	Glossary

01 What the tool is, and what it refuses to be

vulnscan is a service scanner that produces a report. It opens TCP connections to a fixed list of ports, works out what is listening, judges that service against a fixed rule set, and renders the result as HTML, JSON, CSV and PDF. It is a single Python package with no third-party dependencies; nmap is optional and shelled out to when present.

Three design commitments explain almost every decision in the codebase. If you understand these, the rest is detail.

COMMITMENT	WHAT IT MEANS IN CODE
Identify before judging	A port number only selects which rules are *worth running*. A finding fires only when the protocol family matches what the rule expects, or when the port is honestly marked unidentified. See `EXPECTED_FAMILIES` and `_exposure_confidence` (§4).
Evidence over absence	An unanswered port is recorded as an identification gap, not as a clean result: `identification_error`, `coverage.unidentified`, and a "not identified" row in the report (§4, §11).
Reproduce the scan	The report carries the engine, its version, the exact argv, the scan source and platform, the skipped port states, what was *not* tested, and the raw nmap XML is written next to the report (§2, §11).

The numbers that describe it

58	rule IDs: 57 static ones across exposure, TLS, HTTP, FTP and version fingerprints, plus the database-derived `VS-<CVE>` family
21	port-gated exposure rules, each with an expected protocol family
60	default ports (`-p top`); `-p all` is 65,535, `-p 1-1000` and lists/ranges are accepted
9	read-only app-layer probes (Docker, etcd, Kibana, Kubernetes, Prometheus, Elasticsearch, kubelet, RabbitMQ) plus a MongoDB handshake
10	product keys the advisory database can answer for (apache-httpd, nginx, openssh, openssl, mysql, mariadb, postgresql, redis, php, vsftpd)
7 / 10	NSE scripts in the `safe` profile / in `extended`; `off` runs none
304	tests, including a positive and a negative fixture for every rule ID

What it deliberately does not do

- **No exploitation and no authentication.** Nothing sends credentials, nothing writes, nothing escalates. The one place a payload is sent is `--lab-exploits`, which is hard-wired to `127.0.0.1` and only runs against fingerprints it recognises (§6).
- **No UDP or SCTP.** The engine, the probes and the report's coverage text all assume TCP.
- **No version guessing.** If the banner does not reveal a version, no CVE is claimed. CVE matching is only ever as good as the identification behind it, and the report says so in its own limits section.
- **No automatic feed downloads during a scan.** A scan never touches the network except to reach its targets. The advisory database is built only by `--update-db`, which is a separate, explicit command (§9).
- **No bundled nmap.** It is discovered on `PATH` or via `VULNSCAN_NMAP` and invoked as a subprocess. Its licence forbids redistribution, and the point of this project is the report around someone else's detector.

> **Why this shape?** A scanner's reputation is made by what it refuses to claim. Every one of the commitments above exists because the alternative produces confident rubbish: "RDP exposed" on any port 3389, "no findings" on a tarpitted host, or a report nobody can reproduce six weeks later.

02 The pipeline, end to end



STAGE	FUNCTION	DECIDES
Parse	`cli.main`	Saved config vs per-run flags; exit 2 on a bad spec before any packet is sent.
Expand	`targets.parse_targets`, `parse_ports`	Which host/port pairs exist; CIDR larger than 1024 addresses is refused, not truncated.
Resolve engine	`engines.resolve`	nmap or native, and the `fallback_reason` string that ends up in the report.
Scan	`nmap.run` / `native.run`	Which ports are open and, for each, what identity claims exist.
Enrich	`scanner.collect_evidence`	TLS, banner, HTTP, app-probe and product/version facts — the same evidence whichever engine found the port.
Advisory DB	`advisories.AdvisoryDB`	Whether version-range matching is available at all; recorded either way.
Check	`checks.run_checks`	Which findings exist, their severity, their confidence and their stable IDs.

STAGE	FUNCTION	DECIDES
Filter	`report.confirmed_only`	With `--strict`, keeps only `certain` findings and counts the suppressed ones.
Assemble	`report.build_report`	The JSON contract: `metadata`, `scope`, `coverage`, `summary`, `services`, `findings`.
Render	`write_json`, `write_html`, `write_csv`, `pdf.html_to_pdf`	Artifacts on disk, all sharing one file stem derived from `-r`.

> **The two-network-phase rule.** Everything between "services collected" and "findings returned" is pure except two deliberate exceptions: the anonymous-FTP login attempt and the optional TLS-1.0/1.1 handshakes. That is why the checks are cheap to re-run, why the fixtures in `tests/test_rule_matrix.py` can build a `PortService` by hand, and why a report can be regenerated from a saved JSON without touching the network.

03 The data model

Three objects carry the whole tool. Learn their fields and you can read any function in the package without looking anything up.

`PortService` — one open port, with its evidence

Defined in `vulnscan/scanner.py` as a dataclass. It is created by the scan engine and then *mutated* by the evidence pass; after that it is read-only. Checks never write to it.

FIELD	WRITTEN BY	MEANING
`host`, **`ip`**, **`port`**, **`proto`**	scan	The label the user asked for, the resolved address, and the port. `host` stays the hostname so reports read naturally.
`state`	scan	`open`, `closed` or `filtered`. Only `open` services reach the checks.
`service`	scan + evidence	Protocol family name, e.g. `ssh`, `https`, `docker`. Empty means unidentified — never a guess.
`product`, **`version`**	evidence	Extracted from a banner, HTTP `Server` header or protocol handshake. This pair is the only input to CVE matching.
`banner`	evidence	Up to 4 KB of what the service said without being asked (server-first ports) or after one probe.
`probe`	evidence	What was sent to elicit that banner, so the report can be honest about it.
`tls`	evidence	`{ok, version, cipher, cipher_bits, cert{…}}`; `cert` carries subject, issuer, SANs, dates, days remaining, self-signed flag, hostname match.
`http`	evidence	`{status, scheme, path, headers, title, server, body_preview}` from a single GET.
`app`	evidence	Read-only product verification keyed by product: `docker`, `etcd`, `kubelet`, `kibana`, `rabbitmq`, `prometheus`, `elasticsearch`, `kubernetes`, `mongo`.
`source`	scan	Which engine produced the row: `nmap` or `native`.
`nmap_conf`	nmap	nmap's own confidence, 0–10. `≥ 8` is the only value that counts as a strong claim.

FIELD	WRITTEN BY	MEANING
`cpe`, **`extrainfo`**, **`os_matches`**, **`scripts`**, **`rtt_ms`**	nmap	nmap extras: CPE strings, `extrainfo` (often the distro), OS guesses, NSE output, round-trip time.
`identity_source`	scan + evidence	How the current `service` was decided: `port`, `banner`, `http`, `nmap`, or `""` when nothing proved it.
`identity_claims`	scan + evidence	Every claim made about the port, each with its own source. Conflicts are detected from this list, not from the final value.
`identification_error`	scan	Set when the port accepted TCP and then went silent, reset, or nmap called it `tcpwrapped`. Non-empty means "identification gap", not "clean".
`scanned_at`	scan	UTC timestamp, so evidence has an age.

`Finding` — one judgement, with its reasons

Defined in `vulnscan/checks.py`. It is deliberately dumb: text, a severity, a confidence, and pointers to the asset. Everything derived — CVSS, ATT&CK, detection guidance, entry-point flag, provenance — is computed at report time by `report._finding_dict`, so a rule author never has to remember to fill in a score.

FIELD	MEANING
`rule_id`	Stable identity of the rule, e.g. `VS-EXP-DOCKER`. Used as the key for guidance, CVSS vectors, tests and remediation grouping. Advisory findings use `VS-CVE-2024-6387`.
`title`, **`description`**, **`impact`**, **`remediation`**	Prose written for two readers at once: an engineer who must fix it and a manager who must prioritise it.
`severity`	One of `critical`, `high`, `medium`, `low`, `informational`.
`confidence`	`certain`, `firm` or `tentative` — the evidence axis, independent of severity (§8).
`evidence`	The actual bytes or response that justify the finding, truncated to something a human can read. Binary handshakes are described, not dumped.
`asset`	`{host, ip, port, protocol, service}` plus `url` for HTTP findings. This is what ties a finding to a row in the asset inventory.
`cwe`, **`cve`**, **`references`**	CWE IDs, CVE IDs, and links (CWE page, vendor docs, advisory URL).
`cvss_vector`	Only set when an advisory supplied NVD's own vector. Empty means the report derives one (§8).
`id`	Assigned at the end of `run_checks`: `VS-0001` … in severity order, then host/port, so two scans of the same host produce the same numbering.
`status`	`open` today; the field exists so a later workflow can carry `accepted` / `fixed` without a schema break.

The report dict — the JSON contract

`report.build_report` returns a plain dictionary with schema version `1.3`. Everything else in the report layer renders from this structure, so JSON, HTML, CSV and PDF can never disagree about a fact.

KEY	CONTENTS
`report_version`	`"1.3"`. Bumped when a key is added, as `metadata.report_name` was.
`metadata`	Title and report name, scanner name/version, engine, `scan` meta, `advisory_db` meta, scan source host, generated/started timestamps, duration.

KEY	CONTENTS
`scope`	Targets, port count, compact port range, method sentence, and the options actually in force (timeout, concurrency, strict, reference, NSE profile, version intensity).
`coverage`	The provenance block: engine and version, requested engine, fallback reason, argv and shell-joined command, privileged flag, scripts, timeouts, skipped port states, `unidentified[]`, `app_probes[]`, advisory status, authorisation reference, `not_tested[]`.
`summary`	Total findings, counts by severity, open service count.
`services`	One dict per open service. Optional keys (`tls`, `http`, `cpe`, `scripts`, `os`, `app`, `identity_claims`, `identification_error`) appear only when they exist, which keeps the JSON small.
`findings`	One dict per finding, enriched with `cvss`, `attack`, `detection`, `next_step`, `entry_point` and `provenance`.

> **Why derive instead of store?** If CVSS were stored on the finding, changing a curated vector would silently leave old numbers in place, and a rule could be edited into a state where its severity and its score disagree. Deriving it at write time means the vector table is the single source of truth — and `tests/test_cvss.py` pins every vector to the band its rule emits.

04 Identity: ports locate, protocols decide

This is the core of the tool. Everything else is bookkeeping around it.

A port number is a hint about where to look, never a fact about what is there. Port 3389 is *usually* RDP; it is sometimes an HTTPS admin panel, sometimes a honeypot, sometimes an SSH tunnel endpoint. A scanner that reports “RDP exposed” because something answered on 3389 is doing nmap-without-`-sV`, and it is the single largest source of false positives in this class of tool. vulnscan therefore separates **locating** from **deciding**.

The four facts that decide a port's fate

1. Family	`service_family(svc)` maps the identified service name onto a family through `SERVICE_FAMILIES` (52 service names → 31 families). If the name is unknown it falls back to transport facts: an HTTP response means `http`, a completed handshake means `tls`, otherwise `unknown`.
2. Expected	`EXPECTED_FAMILIES[port]` lists the families that make a rule meaningful on that port. Port 2376 expects `{docker, http, tls}`; port 3389 expects `{rdp}` and nothing else.
3. Strength	`identity_source` in `{nmap, http, banner}`, or an nmap claim with `conf ≥ 8` and a real protocol name. `port` is *not* strong.
4. Conflict	Two strong claims that map to different families, ignoring families that all ride HTTP (a Docker API and an HTTP response are the same protocol seen two ways). Conflicts downgrade to `tentative` rather than picking a winner.

The decision itself is eight lines:

```
def _exposure_confidence(svc, expected):
    if identity_conflicts(svc):
        return "tentative"
    if identity_is_strong(svc):
        return "firm" if service_family(svc) in expected else None
    return "tentative"
```

Read the third line carefully, because it is the whole point: a **strong** identity that is **not** in the expected set returns `None`, and `None` means the finding is not created at all. A strong identity in the expected set is `firm`. A *weak* identity — one nothing proved — is `tentative`, which still produces a finding, labelled.

SITUATION	RESULT
3389 answers with an SSH banner	Strong `ssh`; `ssh ≠ rdp` → **no RDP finding**. Nothing claims SSH either: there is no SSH exposure rule at all, only a version fingerprint.
3389 accepts TCP, says nothing	Weak identity → **tentative** RDP finding: "port 3389 is open; the protocol was not confirmed". Uncertain, not silent.
3389 completes a TLS handshake	Strong `tls`; family `tls ≠ rdp` → no RDP finding, and the TLS rules run on their own evidence.
3389 identified as RDP by nmap (`conf 10`)	Strong `rdp`, family matches → **firm**. `--nse extended` adds `rdp-ntlm-info` output as quoted corroboration — same confidence, better evidence.
Redis speaking on 6380	No entry in `EXPOSURE_RULES`, but `_check_redis` is keyed by **family**, so it is still found and judged correctly.
Apache answering on 6379	Family `http`; `_check_redis` returns early for an identified non-Redis service, so no false Redis finding.

What can upgrade a tentative claim

Uncertainty is not permanent — the code looks for proof in this order, each step recorded as its own identity claim:

1. **A curated NSE script answer.** `SERVICE_PROOF_SCRIPTS` maps ports to a script and a marker: `22 → ssh-auth-methods`, `445 → smb-os-discovery` (`OS:` line), `3389 → rdp-ntlm-info` (`Product_Version`, `Target_Name`). A real answer upgrades `tentative → firm`.
2. **An app-layer probe.** A 200 with the product's own markers (Docker's `ApiVersion`, Elasticsearch's `cluster_name`, kubelet's pod list) upgrades to `certain`. A 200 without markers, or a 401/403, proves the product exists but not that it is open → `firm`.
3. **A protocol handshake the scanner can parse.** MongoDB's `listDatabases` reply with `ok: 1` is `certain`; an auth-required error is `firm`.

Unidentified is a first-class outcome

`UNIDENTIFIED_SERVICE_NAMES` is `{"", "unknown", "tcpwrapped"}`. Two paths set `identification_error`:

- **Native:** a connection accepted and then reset/aborted while identifying → `"connection reset during identification"`. The service name is cleared so no port-table guess can stand in for a protocol.
- **nmap:** a `service` element named `tcpwrapped` or `unknown` → `"nmap reported tcpwrapped — the port answered TCP but sent no readable protocol"`. Its `conf` value is ignored, because a high confidence in "I could not tell" is not an identification.

That gap then appears in four places: the console note at the end of the scan, `coverage.unidentified` in the JSON, a row in the report's coverage table, and `not identified` in the asset inventory CSV. The report also never counts it as a clean port.

> **The failure this replaced.** Before this model, `EXPOSURE_RULES` was keyed on the port number alone and fired with `confidence="firm"`. Scanning `-p all` against any host produced a page of confident highs about services that were never there, and a tarpitted port 22 produced nothing at all — the two failure modes that make a scanner untrustworthy at the same time.

05 The two engines: nmap and native

Both engines return the same thing — `list[PortService]` plus a metadata dict — so nothing downstream cares which one ran. The difference is what they can prove and what they cost.

	NMAP ENGINE	NATIVE ENGINE
Finds open ports by	nmap's own connect/syn scan	one `asyncio.open_connection` per host×port
Identifies by	`-sV` probe database, ~2,200 service fingerprints	first-bytes classification plus 5 curated payloads, then a second pass for TLS/HTTP
Can produce	service name, product, version, `extrainfo`, CPE strings, OS matches, NSE output	service name, banner-derived product/version, TLS, HTTP, app-probe proof
Privileges	works unprivileged with `-Pn -sV`; richer with raw sockets	none beyond outbound TCP
Reported as	`engine: "nmap"`, version from the XML	`engine: "native"`, version `vulnscan 0.1.0`

Engine resolution

`engines.resolve()` is deliberately dull: `native` never looks for nmap; `nmap` raises `EngineUnavailable` rather than silently scanning with something else; `auto` prefers nmap and records why it fell back. That reason is printed, stored as `coverage.fallback_reason`, and shown in the report's coverage table — a scan that quietly used the weaker detector would be lying about its own evidence.

The nmap command line

Built by `build_argv`, passed to `subprocess.run` as a list. No shell, so nothing in a target or option can be interpreted as a command; the port list is compacted into ranges (`-p-` when the whole range is requested).

```
nmap  -sV [--version-intensity N]
      -Pn                  # treat targets as up; ICMP is often filtered
      -n                   # no reverse DNS; names come from the targets
      -T4 --max-retries 2 --host-timeout 300s
      [--script ssl-cert,http-title,… --script-timeout 30s]
      -oX - -p 22,80,443 … host…
```

The whole subprocess has its own ceiling (`process_timeout_s=1800`) separate from the per-host timeout, so a wedged nmap cannot hang a scan forever. The XML comes back on stdout and is parsed with `ElementTree`; the raw text is stashed in `meta["_raw_xml"]`, which the CLI pops and writes next to the report.

What `parse_xml` keeps, and what it refuses

- Only `state=open` ports become services; every other state is counted into `skipped_ports` (`closed`, `filtered`, …) and surfaces in coverage. A filtered port is never mistaken for a clean one.
- `tunnel=ssl` + `http` is renamed `https`, because nmap reports the pair as `ssl/http`.
- The host label is the name the user asked for, then the PTR record, then the address — so a report says `scanme.org:22`, not `45.33.32.156:22`.
- **`tcpwrapped` and `unknown` are not identifications.** The service name is dropped, `identification_error` is set, and `identity_source` becomes `""`. nmap's `conf` is ignored in that case: a high confidence in "I could not tell" is still not a protocol.

- NSE script output is kept only when it is a real answer — empty runs, `Script execution failed` and `ERROR:` lines are dropped rather than printed as evidence.

Why nmap's own detection is not simply trusted

The nmap engine still runs `_enrich()`: for every open TCP port it calls the same `collect_evidence` the native engine uses. When nmap's name is *weak* (`conf < 8`, `unknown`, or a trailing `?`), the evidence pass is allowed to name the port from what the service actually said (`guess_names=True`). When nmap's name is strong, its product/version is kept and the evidence pass only fills gaps. This is how a host that nmap calls "maybe SSH" gets corrected by an actual banner, and how a verified `OpenSSH 9.6p1` keeps nmap's precision instead of being overwritten by a regex that saw less.

The native engine, connection by connection

`probe_port` is the only place a port is identified, and it is written to spend as few connections as possible. The order is:

1. **TLS-first ports** (`TLS_PORTS`: 443, 465, 636, 993, 995, 2376, 5671, 6443, 8443, 8883, 9443) go straight to a handshake. A refused connection ends the probe as `closed`, a timeout as `filtered` — neither gets a second attempt. On success the same socket is reused for the HTTPS request (`TLS_HTTP_PORTS`).
2. **Everything else** gets one plain TCP connect. Server-first services (FTP, SSH, SMTP, MySQL, VNC … — `SERVER_FIRST_PORTS`) are read immediately; client-first services are sent a read-only probe on *that same socket*.
3. **First-bytes classification** runs on whatever arrived. The classifier checks binary handshakes first (MySQL's protocol byte, PostgreSQL's length-prefixed message, MongoDB's opcode), then textual markers, most specific first: `ssh-`, `rfb 00`, `redis_version`, `-err unknown command`, `stat pid` / `stat version`, `220 (vsftpd` / `proftpd` / `pure-ftpd`, `+ok`, `* ok`, `amqp`, `mysql_native_password`, `password authentication failed`, `\xffsmb`, `http/`.
4. **A bare `220` is ambiguous** — FTP and SMTP both greet that way — so it is recorded as `ftp/smtp` and never allowed to overrule a port-based guess. It may still name a port that was otherwise silent.
5. **If nothing plaintext answered**, one TLS handshake is tried as a last resort. That is how an HTTPS service squatting on an unusual port is identified instead of being mislabelled from the port table.
6. **Only then** may the port table supply a name, and `identity_source` is set to `port` — which is exactly the marker the checks treat as *not proven*.

The connection budget is asserted by `tests/test_sniffer.py`: one connection for SSH, one for HTTP, one for a waiting service that must be asked, and two for HTTPS on an unusual port (handshake, then a separate request). App-layer probes and the FTP login attempt are the only extra connections, and both are deliberate, read-only requests.

CONSTANT	WHAT IT HOLDS
`PORT_SERVICES`	58 port → name entries. A guess, used only when nothing on the wire named the service.
`PROBES`	5 payloads for services that wait to be addressed: SMTP `EHLO`, Redis `INFO server`, memcached `stats`, ZooKeeper `ruok` (and 587 as SMTP submission).
`active_probe()`	The same map, plus a MongoDB `isMaster` OP_QUERY and the classic RDP X.224 negotiation request for 3389.
`HTTP_PORTS` / **`NON_HTTP_PORTS`**	Where an HTTP GET is worth trying first, and where it is pointless noise (SSH, RDP, MySQL …). HTTP is still attempted on a port that said nothing at all.

> **Why keep two engines at all?** nmap is better at identification and has version probe data this project will never reproduce. The native engine exists so the tool works on a machine with nothing but Python — and so every report can be reproduced by someone who does not have nmap installed. Neither is a shim around the other: they share the evidence pass, the checks and the report.

06 Evidence: what is actually said on the wire

All evidence collection lives in `scanner.py` and is called from exactly one place per engine (`collect_evidence` for nmap, `probe_port` for native). The rule for everything in this section: **read only**. No credentials, no writes, no exploitation.

COLLECTOR	WHAT IT DOES AND WHY IT IS BOUNDED
`grab_banner`	Reads at most 4 KB, decodes as latin-1 (so binary handshakes do not raise), returns the first 1 KB plus a note of what was sent. Timeouts and read failures return empty strings with a reason, never an exception.
`http_fetch`	One `GET /` (or a product path), explicit `Host` header including a non-default port, UA `vulnscan/0.1`, and a 64 KB body cap with a grace period after the headers. Returns status, headers, title and a body preview — nothing is followed, retried or authenticated.
`tls_session` / `probe_tls`	Handshake with verification disabled (the certificate is *analysed*, not trusted), then version, cipher, bits and a decoded certificate: subject, issuer, SANs, notBefore/notAfter, days remaining, self-signed, hostname match. DER decoding uses CPython's `_test_decode_cert`, a private API — a known, contained risk: failure returns `{}` and the TLS checks fall back to what the handshake itself showed.
`supports_tls_version`	Pins `minimum_version = maximum_version` and tries a handshake. Only called with `--legacy-tls`, because it is two extra connections per TLS service and most engagements do not need it.
`probe_app_layer`	One GET against a product-specific endpoint, recorded as `svc.app[product]` with status, body preview and a `confirmed` flag. A 200 whose body carries the product's markers is proof; a 200 without them, or a 401/403, proves presence with auth in front.
`probe_mongo`	A hand-rolled OP_QUERY for `isMaster` and `listDatabases` built with `struct` and a minimal BSON encoder. It parses the reply just far enough to read `ok` and the version. No credentials are ever sent, and `listDatabases` on a server without auth is the difference between “reachable” and “open”.
`_extract_product`	Regex fingerprints for the cases that matter: `SSH-2.0-OpenSSH_9.6p1`, `220 (vsFTPd 3.0.3)`, SMTP greetings and `Postfix` / `Exim` / `sendmail`, MySQL's handshake version string, Redis `redis_version:…`, memcached `STAT version …`, and `Server: nginx/1.18.0 (Ubuntu)` → `(nginx, 1.18.0)`.

The nine app-layer probes

PORT	PRODUCT	ENDPOINT	PROOF LOOKS LIKE
2375 / 2376	docker	`/version`	body contains `docker` and `apiversion`
2379	etcd	`/health`	`health`
5601	kibana	`/api/status`	`status`, `version`
6443	kubernetes	`/version`	`gitversion`, `major`
9090	prometheus	`/api/v1/targets`	`activetargets`, `success`
9200	elasticsearch	`/_cluster/health`	`cluster_name`, `status`

PORT	PRODUCT	ENDPOINT	PROOF LOOKS LIKE
10250	kubelet	`/pods`	`items`, `podlist`, `apiversion`
15672	rabbitmq	`/api/overview`	`rabbitmq_version`, `management_version`
27017	mongo	BSON handshake	`ok: 1` from `listDatabases`, or an auth error

Probes are skipped unless the port is in the table *and* the service looks HTTP-ish (or the port's own service already matches the product), so an SSH daemon never receives an HTTP request. Acceptable statuses are 200, 401, 403 and 404; anything else is discarded rather than guessed at.

The one place a payload is sent on purpose

`vulnscan/lab_exploits.py` runs fixed, non-destructive proof payloads — an anonymous FTP login, a Redis `INFO`, a `/version` on a Docker API, a path-traversal read against vsftpd 2.3.4 — but only for `--lab-exploits`, only to `127.0.0.1`, only when a lab fingerprint matches, and with a response cap. The module has no host argument by design: there is no way to aim it at a real target.

07 The rule catalogue

Fifty-seven static rule IDs live in `vulnscan/checks.py`, and a fifty-eighth family — `VS-<CVE>` — is generated from the advisory database. They are grouped here by *what gates them*, because that is the thing you need to know when deciding whether a finding is trustworthy.

`run_checks` runs three families per service in a fixed order — exposure, HTTP, fingerprints — then the two network-dependent families across the whole service list (anonymous FTP, then TLS with optional legacy handshakes). Findings are then sorted by severity, host, port and rule ID, and given `VS-0001`… IDs, so numbering is reproducible for the same input.

A. Port-gated exposure (21 ports, 20 IDs)

Fires only when the protocol family matches the port's `EXPECTED_FAMILIES`; otherwise it is suppressed (contradicted) or downgraded to tentative (unidentified). See §4.

PORT	RULE ID	EXPECTED FAMILY	SEVERITY
23	`VS-EXP-TELNET`	telnet	high
2323	`VS-EXP-TELNET-ALT`	telnet	high
3389	`VS-EXP-RDP`	rdp	high
445	`VS-EXP-SMB`	smb	high
5900, 5901	`VS-EXP-VNC`	vnc	high
2376	`VS-EXP-DOCKER-TLS`	docker, http, tls	high
2379	`VS-EXP-ETCD`	etcd, http	high
10250	`VS-EXP-KUBELET`	kubelet, http	high
5985	`VS-EXP-WINRM`	winrm, http	high
873	`VS-EXP-RSYNC`	rsync	medium
1433	`VS-EXP-MSSQL`	mssql	medium
1521	`VS-EXP-ORACLE`	oracle	medium

PORT	RULE ID	EXPECTED FAMILY	SEVERITY
2049	`VS-EXP-NFS`	nfs	medium
3306	`VS-EXP-MYSQL`	mysql	medium
5432	`VS-EXP-POSTGRES`	postgres	medium
5601	`VS-EXP-KIBANA`	kibana, http	medium
15672	`VS-EXP-RABBITMQ`	rabbitmq, http	medium
9090	`VS-EXP-PROMETHEUS`	prometheus, http	low
9201	`VS-EXP-ADMIN`	http	low
6443	`VS-EXP-K8S-API`	kubernetes, http, tls	informational

B. Evidence-gated services (keyed by protocol, not port)

RULE	FIRES WHEN	SEV / CONFIDENCE
`VS-EXP-REDIS-NOAUTH`	`INFO` returned `redis_version` without `AUTH`	critical / certain
`VS-EXP-REDIS-AUTH`	Redis answered `NOAUTH` / “authentication required”	medium / firm
`VS-EXP-REDIS`	Looks like Redis but the banner revealed neither	medium / tentative
`VS-EXP-MEMCACHED-NOAUTH`	`stats` returned `STAT …`	critical / certain
`VS-EXP-MEMCACHED`	Memcached family, no stats answer	medium / firm or tentative
`VS-EXP-DOCKER`	Docker family or port 2375, not contradicted by another strong identity	critical / certain, firm or tentative by proof
`VS-EXP-MONGODB`	Mongo family or 27017; `listDatabases` raises it to certain	high / certain, firm or tentative
`VS-FTP-ANON`	`USER anonymous` answered `230`	high / certain
`VS-FTP-CLEARTEXT`	FTP family or port 21, no TLS observed	low / certain or tentative

The `contradicted()` helper is what keeps these honest: a strong identity in a different family suppresses the check entirely, so an HTTP server on 11211 is not called memcached and an SSH daemon on 27017 is not called MongoDB.

C. TLS (7 rules, only when a handshake succeeded)

RULE	CONDITION	SEVERITY	CONFIDENCE
`VS-TLS-EXPIRED`	Certificate past `notAfter`	high	certain
`VS-TLS-NOT-YET-VALID`	`notBefore` in the future (clock skew)	medium	certain
`VS-TLS-EXPIRING`	Fewer than 14 days remaining	medium	certain
`VS-TLS-SELF-SIGNED`	Subject equals issuer	low	certain
`VS-TLS-HOSTNAME`	No SAN covers the scanned host	medium	firm
`VS-TLS-WEAK-CIPHER`	Negotiated cipher contains NULL, EXPORT, RC4, 3DES or anon	high	certain
`VS-TLS-LEGACY`	A pinned TLS 1.0/1.1 handshake succeeded (`--legacy-tls`)	medium	firm

When nmap also ran `ssl-cert`, its output is appended as corroboration to the certificate findings rather than creating a second finding for the same fact.

D. HTTP (16 rules, only when a response was parsed)

RULE	LOOKS AT	SEVERITY	CONFIDENCE
VS-HTTP-CLEARTEXT	HTTP served where HTTPS is expected	informational	certain
VS-HTTP-SERVER-VERSION	`Server:` discloses a version	low	certain
VS-HTTP-POWERED-BY	`X-Powered-By:` present	low	certain
VS-HTTP-COOKIE-NO-SECURE	`Set-Cookie` without `Secure`	medium	firm
VS-HTTP-COOKIE-NO-HTTPONLY	`Set-Cookie` without `HttpOnly`	low	firm
VS-HTTP-DIR-LISTING	Index page is a directory listing	medium	certain
VS-HTTP-STACKTRACE	Framework debug output in the body	medium	certain
VS-HTTP-ES-OPEN	Port 9200, 200 with `cluster`	critical	certain
VS-HTTP-ES-EXPOSED	Port 9200, 401/403	medium	firm
VS-HTTP-DOCKER-VERSION	2375/2376, 200 with Docker markers	critical	certain
VS-HTTP-ADMIN-OPEN	Console on 5601/15672/9090/8500/8983/5985 answering 200	high	firm
VS-HTTP-HSTS	Missing Strict-Transport-Security	medium	certain
VS-HTTP-CSP	Missing Content-Security-Policy	medium	certain
VS-HTTP-XFO	Missing X-Frame-Options / frame-ancestors	low	certain
VS-HTTP-XCTO	Missing X-Content-Type-Options	low	certain
VS-HTTP-REFPOL	Missing Referrer-Policy	informational	certain

E. Version fingerprints (5 rules)

RULE	MATCH	SEV / CONFIDENCE
VS-CVE-2011-2523	vsftpd **2.3.4** exactly — the release shipped with a backdoor	critical / certain
VS-CVE-2021-41773	Apache httpd **2.4.49** — path traversal / RCE	critical / firm
VS-CVE-2021-42013	Apache httpd **2.4.50** — the incomplete fix	critical / firm
VS-CVE-2024-6387	OpenSSH **8.5p1–9.7p1** — regreSSHion	medium / tentative
VS-EOL-SOFTWARE	Product/version past its support window	medium / firm

These are hand-curated because they carry context an advisory row cannot: what the defect actually is, and whether a version banner is enough to claim it. regreSSHion is **tentative** on purpose — most distributions backported the fix without changing the upstream version, so a banner match alone is a lead, not a conclusion.

F. Advisory-derived findings

`VS-<CVE>` findings come from the SQLite database (§9) and are de-duplicated against the curated rules by CVE, so a matched CVE never appears twice on one service. Severity is NVD's own band; confidence is `firm` for an

upstream version in range and `tentative` when a distro package revision was detected, because a backport may already have fixed it.

> **One property worth remembering:** every check is a pure function of one `PortService` (plus, for FTP and legacy TLS, one explicit network call). There is no shared mutable state between checks, no ordering dependency, and no rule that can suppress another rule's finding. That is what makes the rule matrix (§13) possible.

08 Severity, confidence and CVSS

Three axes, deliberately independent, because collapsing them is how reports lie. A finding can be *high severity and tentative* (a possible RDP on a silent port), or *low severity and certain* (a missing Referrer-Policy).

AXIS	VALUES	QUESTION IT ANSWERS
Severity	critical, high, medium, low, informational	How bad is it if true?
Confidence	certain, firm, tentative	How sure are we that it is true?
CVSS	0.0–10.0 with a vector	What does the industry's scoring say, and from which vector?

What each confidence level has actually proved

CONFIDENCE	PROVENANCE IN THIS CODEBASE
certain	The service answered the question itself: redis `INFO` without auth, memcached `stats`, an anonymous FTP `230`, a Docker `/version` with markers, a Mongo `listDatabases` reply, a certificate the scanner parsed, an HTTP response it read.
firm	The protocol identity is proven but the consequence is inferred: a Redis that demands auth, a console returning 401/403, a distro-backported version match, a hostname mismatch, a version-only CVE match on a banner.
tentative	Nothing proved the protocol; the port number is the only evidence. Also used when two strong identities disagree, and for distro package revisions.

`--strict` keeps only `certain` findings and reports how many it suppressed in the coverage block — the count matters, because a strict report with silent suppression would be worse than a normal one.

> **Tentative does not mean "ignore".** A tentative finding says "something is listening here and we could not prove what; here is what it would mean if the port means what it usually means". On a real engagement that is exactly the list you take into the next phase. The only identities that produce *no* finding are strong ones that contradict the rule — those are the false positives this design exists to remove.

CVSS: the vector is the source of truth

`vulnscan/cvss.py` implements the FIRST CVSS 3.1 base formula — attack vector, attack complexity, privileges required, user interaction, scope, and the three impact metrics — with the standard `Roundup` to one decimal. It does not store scores anywhere: it stores vectors and computes.

PRECEDENCE	SOURCE	USED WHEN
1	`CVSS:3.x/…` from the advisory row	An NVD-sourced CVE match. Reported as `source: "nvd"`, and a 3.0 vector is labelled 3.0 rather than silently upgraded.

PRECEDENCE	SOURCE	USED WHEN
2	`CURATED_VECTORS[rule_id]`	Every built-in rule has one (57 entries), written to match its severity and its real-world conditions.
3	`BAND_VECTORS[severity]`	Any rule without a curated vector — a floor, so no finding renders without a score.

EXAMPLE RULE	VECTOR	SCORE	BAND
VS-EXP-DOCKER	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`	9.8	critical
VS-EXP-REDIS-NOAUTH	same	9.8	critical
VS-EXP-TELNET	`AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`	8.1	high
VS-TLS-EXPIRED	`AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`	7.4	high
VS-HTTP-HSTS	`AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`	5.3	medium
VS-EXP-K8S-API	`AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:N/A:N`	0.0	informational

The score is derived, and so is the band shown beside it: `band_for(score)` uses the same thresholds as the severity list (9.0 / 7.0 / 4.0 / 0.1). If a curated vector scored outside its rule's band, that is a bug in the rule — and `test_every_rule_vector_lands_in_its_rule_band` fails the build rather than letting the report paper over it at render time.

09 The offline advisory database

A scan never touches the network except to reach its targets, so CVE matching cannot depend on a live API. `vulnscan/advisories.py` solves this by turning NVD's CVE JSON into a small SQLite file that is built once and queried locally, read-only, on every scan. Standard library only: `sqlite3`, `urllib`, `gzip`.

Source	`https://nvd.nist.gov/feeds/json/cve/2.0/nvdcve-2.0-{year}.json.gz`, one gzip file per year
Default years	the last six (`--db-years 2021,2022` overrides)
Default path	`$XDG_CACHE_HOME/vulnscan/advisories.sqlite3` (usually `~/.cache/vulnscan/`), overridable with `$VULNSCAN_DB` or `--db`
Build command	`python3 -m vulnscan --update-db` — explicit, and it exits without scanning
Query mode	`file:…?mode=ro`; a scan can never create or modify the database

What one CVE row becomes

NVD describes affected products as CPE criteria nested in configuration nodes, each with an optional version range. `rows_from_nvd` flattens that into rows keyed by *product*, not by CPE:

- `_CPE_TO_KEY` maps the 13 (vendor, product) pairs worth answering for onto 10 product keys: `apache-httpd`, `nginx`, `openssh`, `openssl`, `mysql`, `mariadb`, `postgresql`, `redis`, `php`, `vsftpd`. Everything else in NVD is skipped by design — a database that claims coverage it cannot verify is worse than a small one.
- A range becomes `introduced` / `fixed` with inclusive flags taken from NVD's own `versionStartIncluding`, `versionStartExcluding`, `versionEndIncluding` and `versionEndExcluding`.
- A single affected release encoded in the CPE string (`…:2.4.49:…` with no range) becomes `exact_version`.
- A bare wildcard with no range and no fixed release is **skipped**: “all versions, no version information” cannot be graded honestly, so it is not imported.

- Severity comes from NVD's score (≥9 critical, ≥7 high, ≥4 medium, >0 low), falling back to NVD's label, then to medium. The CVSS vector string is stored verbatim and later becomes the finding's `cvss_vector`.

The schema

```
CREATE TABLE meta (key TEXT PRIMARY KEY, value TEXT);
CREATE TABLE advisory (
    cve             TEXT NOT NULL,
    product         TEXT NOT NULL,
    introduced      TEXT,      -- versionStartIncluding / Excluding
    start_inclusive INTEGER NOT NULL DEFAULT 1,
    fixed           TEXT,      -- versionEndExcluding / Including
    end_inclusive    INTEGER NOT NULL DEFAULT 0,
    exact_version    TEXT,     -- a single release, no range
    severity         TEXT NOT NULL,
    cvss             REAL,
    vector           TEXT,     -- CVSS:3.1/… verbatim
    summary          TEXT,
    url              TEXT
);
CREATE INDEX advisory_product ON advisory(product);
```

`meta` carries `schema_version`, `built_at`, `advisories`, `years` and `source`. Those values are what the report prints as "X CVEs, built on …" and what the coverage table quotes when the database is missing.

Matching a detected version

`AdvisoryDB.lookup(product, version)` fetches the rows for one product and evaluates each range in Python:

```
exact_version set        →  parse(version) == parse(exact)
introduced set           →  version >  introduced   (or >= if start_inclusive)
fixed set                →  version <  fixed        (or <= if end_inclusive)
```

Two details matter for correctness:

- **Version comparison is numeric groups, not string comparison.** `parse_version` returns `(9, 6, 1)` for `9.6p1` and `(2, 4, 49)` for `2.4.49`, and `_pad` zero-fills the shorter tuple so `1.18` equals `1.18.0`. It is deliberately not a full package-version comparison — it matches the ordering NVD's ranges assume for these ten products.
- **One advisory per CVE.** NVD often lists several ranges for one CVE; the lookup keeps the row with the highest CVSS and returns sorted results. The report must not repeat the same CVE because two CPE ranges matched.

Distributions and backports: the honest part

`split_version` separates an upstream version from a package revision: `1.18.0-1ubuntu1` → `("1.18.0", "ubuntu")`, `-2.el9` → red hat, and when the revision says nothing it sniffs the hints nmap and the `Server` header provide. The distro name changes the finding, not the match:

- Upstream version in range → `firm`.
- A distro package revision detected → `tentative`, plus a sentence in the description explaining that distributors routinely backport fixes without changing the upstream version, and a note in the evidence that backport status is unknown.

This is the single most valuable piece of restraint in the tool. A scanner that reports "nginx 1.18.0 is vulnerable" on an Ubuntu host with the security patch applied is not being thorough; it is being wrong in a way that costs the reader hours.

Building and replacing it

`build_database` writes to a temp file in the destination directory, then `os.replace`s it into place — an interrupted download can never leave a half-written database where a scan will read it. A missing or corrupt file is not an error at scan time: `AdvisoryDB.available` is simply false, and the report says CVE matching used the built-in fingerprints only.

> **Freshness is yours to manage.** The database is a snapshot. The report records `built_at` in `metadata.advisory_db` and in the coverage table, so a reader can see how old the CVE knowledge was. Rebuild monthly, or before any engagement where version matching will be quoted.

10 Guidance: ATT&CK, detection, next step

A severity tells a reader how bad something is. It does not tell the blue team what to watch for or the operator what to try next. `vulnscan/guidance.py` adds three fields to every finding, keyed by `rule_id`:

FIELD	CONTENT
`attack`	MITRE ATT&CK technique IDs with names, chosen from 36 named constants such as `T1190` Exploit Public-Facing Application, `T1021.001` RDP, `T1552.007` Container API, `T1610` Deploy Container. Labels for the report, not a claim of observed behaviour.
`detection`	The telemetry that would show someone using the exposure — specific log lines, firewall events, TLS handshake patterns.
`next_step`	A read-only confirmation an operator can run inside the agreed scope, e.g. re-issue `INFO` to see whether `CONFIG SET` is accepted.

There are 57 entries. Two fallbacks make sure a finding is never rendered with empty guidance: `VS-CVE-*` gets a CVE-family default, and anything else gets the generic default. `provenance.guidance` in the JSON records which one applied — `rule`, `cve-family` or `default` — so it is visible when the tool is speaking generically.

Entry points

`is_entry_point(finding)` answers the red-team question: could this be used without first winning a credential? Nine rules qualify (anonymous FTP, unauthenticated Redis/memcached/MongoDB, the Docker API over HTTP or TLS-detectable, an open Elasticsearch, an open admin console, kubelet), plus any `VS-CVE` that is high or critical *and* not tentative. A distro-backport guess is explicitly not an entry point until someone verifies it.

11 The report: JSON, HTML, CSV, PDF

All four artifacts render from the one dict described in §3, so they cannot disagree about a fact. JSON is the contract; HTML is the deliverable; CSV is for the ticketing system; PDF is the same HTML printed.

The HTML report's nine sections

SECTION	WHAT IT ANSWERS
1. Scope and authorisation	Targets, port range, method, engagement reference — the part a client checks first.
2. Coverage and provenance	Engine and version, exact command, privileges, NSE profile, skipped port states, unidentified services, app probes run, advisory database and its build date, the “not tested” list.
3. Executive summary	KPI cards, a stacked severity bar, filter chips, the worst findings, and a prioritised remediation table grouped by rule (sorted by how many findings one action clears).

SECTION	WHAT IT ANSWERS
4. Asset inventory	One row per open service: service, product, version, CPE, TLS, how it was identified.
5. Per-host rollup	One page per host: what runs there, what is wrong, what to fix first.
6. Findings	Severity sections; each finding is a card with badge row, evidence in a code block, ATT&CK mapping, detection and next step.
7. Recommendations	The actions, deduplicated across findings.
8. Methodology and limits	How identification worked, and the seven things the scan did not test.
9. Appendix — NSE output	Raw script output, so a reader can re-derive a conclusion.

Printing is treated as a first-class output: A4 page setup, denser type, two finding cards per page, page breaks before per-host blocks, no shadows or interactive chips. The same 90-finding lab scan rendered 193 pages before that tuning and 101 after — the same content, without half-empty pages.

CSV: the asset inventory, fourteen columns

```
host, ip, port, protocol, service, product, version, cpe,
identified_by, tls, state, findings, critical, high
```

One row per open service. `identified_by` is `identity_source`, which is how a reader can tell a proven service from a port-table guess; the last three columns are counts joined from the findings, so the sheet can be sorted by risk without opening the HTML.

File naming, and why `-r` matters

`report_stem` turns the report name into a slug: Unicode NFKD-folded to ASCII, lowercased, every run of non-alphanumerics to a single hyphen, trimmed, capped at 60 characters, with `vulnscan-` in front and the timestamp behind. `-r "ACME Q1 external"` yields `vulnscan-acme-q1-external_2026-11-06_120000.{json,html,nmap.xml}` and a PDF of the same stem. No name at all yields the old `vulnscan-report_<stamp>`.

PDF without a PDF library

`pdf.html_to_pdf` finds Microsoft Edge first, then Chrome, then checks `PATH` for `chrome` / `chromium`, and runs `--headless --print-to-pdf` against a `file://` URL. That gives a pixel-faithful render of the styled HTML with no third-party package, and a clear error message when no browser exists. The default destination is the Downloads folder (falling back to Desktop, then home), overridable with `--pdf-dir` or saved config.

Raw evidence for reproducibility

When the nmap engine ran, its XML is written beside the report as `<stem>.nmap.xml` and its path recorded in `metadata.scan.raw_xml`. It is never embedded in the report itself: it is evidence a reviewer can re-parse, not a payload to ship to a client. Together with `coverage.command`, this is what lets someone else reproduce a finding months later.

12 The CLI surface and its exit codes

Arguments are parsed once in `cli.build_parser`, in report order: *what to scan → how to detect → what to write*. Every flag below maps onto a specific piece of the pipeline, which is why this table doubles as a table of contents for the code.

GROUP	FLAG	EFFECT INSIDE THE TOOL
Targets	`-p, --ports`	`parse_ports` : `top` (60), `all` (65,535) or explicit list/ranges.
	`-t, --timeout`	Per-probe timeout (default 2.5 s), threaded into `probe_port`, `run_checks` and the TLS handshakes.
	`-c, --concurrency`	Semaphore width for native scans (default 256); nmap's enrichment pass caps itself at `min(32, c)`.
Detection	`--engine`	`auto` / `nmap` / `native`. Recorded as `requested_engine` in coverage.
	`--nse`	`off` / `safe` (7 scripts) / `extended` (10). Never vuln, exploit, brute or DoS categories.
	`--version-intensity`	2–9, passed straight to nmap `-sV`. Unset means nmap's own default.
	`--legacy-tls`	Adds two pinned handshakes per TLS service; the only optional extra network cost.
CVE data	`--db`	Advisory database path for this scan (`$VULNSCAN_DB` or the cache default otherwise).
	`--update-db`	Download NVD feeds, rebuild, exit. Requires no target.
	`--db-years`	Years to include when rebuilding; default is the last six.
Report shape	`--strict`	Keep only `certain` findings; the suppressed count lands in coverage.
	`-r, --report-name`	Title, `metadata.report_name`, and the file stem of every artifact.
	`--reference`	Authorisation/SOW/ticket id, printed in the scope section and stored in coverage.
	`--lab-exploits`	Run the loopback-only proof checks; refuses any target argument.
Output	`-o, --output-dir`	Where HTML/JSON/CSV/raw XML land (default `out`).
	`--pdf-dir`	Where the PDF lands (default Downloads).
	`--json | --csv | --html | --pdf [PATH]`	Optional path override; bare flag means “automatic location”.
	`--no-html | --no-json | --no-pdf`	Opt out of an artifact that is otherwise written by default.
	`-q, --quiet`, **`--no-color`**	Suppress progress and ANSI colour; findings still print.
Config	`--set-output-dir`, `--set-pdf-dir`	Persist a default destination, then exit.
	`--show-config`, **`--reset-config`**	Print or clear the saved settings, then exit.
	`--version`	`vulnscan 0.1.0`.
	Precedence	Per-run flag > saved config (`~/.vulnscan/config.json` , or `$VULNSCAN_CONFIG_DIR`) > built-in default. Only known keys are honoured, so a hand-edited file cannot inject settings.

Exit codes

0	Scan completed with no findings — safe to gate CI on.

1	Scan completed with at least one finding.
2	Bad target/port spec, unknown engine, unavailable nmap with `--engine nmap`, a failed feed download, or `--lab-exploits` given targets. No report is written.

13 Testing: how 304 tests keep the rules honest

Twelve files, 171 test methods, 304 discovered tests — the difference is `subTest`: the rule matrix expands one method into 116 cases, and a few table-driven tests do the same. **No test touches a real network target**; every one stands up a fake server, builds a `PortService` by hand, or feeds XML/JSON/fixtures straight into a parser.

FILE	METHODS	WHAT IT PINS DOWN
`test_rule_matrix.py`	4 → 120	A positive and a negative fixture for every one of the 58 rule IDs, plus four coverage checks: every rule has two cases, the matrix covers every ID, severity matches the rule, and a missing method fails the build. **You cannot ship a rule without a test.**
`test_nmap_engine.py`	35	XML parsing, port-list compaction to `-p-`/ranges, argv construction, `tcpwrapped` and weak-confidence handling, script-output filtering, NSE profiles, engine resolution and fallback.
`test_report_sections.py`	30	Section order, scope and not-tested list, coverage command/privileges/scripts, a CVSS vector on every card, per-host page blocks, entry-points-first ordering, the raw-XML reference, and the CSV's shape, quoting and empty-scan behaviour.
`test_identity_rules.py`	24	The identity model itself: conflicts, HTTP-riding families, strong vs weak claims, exposure gating.
`test_advisories.py`	19	NVD ingestion, exact versions, range inclusivity, wildcard skipping, dedup per CVE, distro parsing, atomic build.
`test_guidance.py`	14	Every rule has guidance; ATT&CK IDs are well formed; fallbacks fire for CVE and unknown rules.
`test_cvss.py`	12	Known base scores, scope change, malformed vectors, and the invariant that every curated vector lands in its rule's band.
`test_report_name.py`	10	`-r` propagation: title, JSON keys, HTML escaping, slug edge cases, PDF stem, real files on disk.
`test_unidentified.py`	9	The reset path, `tcpwrapped` parsing, strong-claim rules and the report rows for unidentified services.
`test_sniffer.py`	8	First-bytes classification and connection counts (1 per port, 2 for HTTPS on an odd port).
`test_probes.py`	4	App-probe semantics: 200 with markers, 200 without, 401, and the wrong product on the right port.
`test_lab_exploits.py`	2	The loopback-only proof checks stay loopback-only and read-only.

How a rule is tested

A case is a `PortService` built with the minimum evidence a rule needs, then an assertion on what `run_checks` returns:

```
# positive: the port answers like Docker, so the finding must appear
DockerOn2375().expect("VS-EXP-DOCKER", severity="critical", confidence="certain")
# negative: same port, an HTTP server with no Docker markers
PlainHttpOn2375().expect_none("VS-EXP-DOCKER")
```

The negative case is the important half. It is what stops the false positives described in §4 from coming back, and it is the reason a new rule costs two fixtures rather than one.

The live rig

`tests/fake_services.py` starts deliberately vulnerable services on loopback (anonymous FTP, unauthenticated Redis, an open Docker API, a directory listing, a self-signed HTTPS endpoint, and more), and `tests/serve_report.py` serves a generated report at `127.0.0.1:8622` for visual review. Both are development tools, not part of the package.

14 Recipes: extending the tool safely

Each recipe lists the files to touch and the checklist that means it is actually done. The checklist is not bureaucracy — the coverage tests will fail on a missing piece, which is the point.

1. Add a port-gated exposure rule

1. `checks.py → EXPOSURE_RULES[port]`: the 7-tuple `(rule_id, title, severity, _cwe(…), description, impact, remediation)`.
2. `checks.py → EXPECTED_FAMILIES[port]`: the family set that makes the rule meaningful. This is what stops it firing on a contradicted port.
3. `checks.py → SERVICE_FAMILIES`: add the service name(s) if the protocol is new.
4. `guidance.py → GUIDANCE[rule_id]`: `_g((T…,), detection, next_step)`.
5. `cvss.py → CURATED_VECTORS[rule_id]`: a vector whose score lands in the declared band.
6. `tests/test_rule_matrix.py`: a positive case and a negative case.
7. `scanner.py → PORT_SERVICES`: only if the fallback guess should know the port.

Done when: the matrix passes, `test_every_rule_vector_lands_in_its_rule_band` passes, and `test_guidance` finds an entry for the new ID.

2. Add an app-layer probe

1. `scanner.py → APP_PROBES[port] = (product, path)` and `APP_MARKERS[product] = (…)` — lowercase substrings unique to that product's response.
2. `checks.py → APP_PROOF_PORTS[port] = product`, so the check can upgrade confidence from the result.
3. If the port must never receive an HTTP request otherwise, check `NON_HTTP_PORTS` / `HTTP_PORTS`.
4. `tests/test_probes.py`: 200-with-markers, 200-without, 401/403, and wrong product.

Done when: a 200 with markers produces `certain`, a 401 produces `firm`, and a different product on the same port produces nothing.

3. Add a curated CVE fingerprint

1. `checks.py → check_fingerprints`: match on `svc.product` and an exact `ver` tuple; set `cve=[…]` and a reference URL.
2. Choose confidence honestly: `certain` only when the version is unambiguous and no backport is plausible, otherwise `firm` or `tentative`.
3. Matrix cases; the negative case is a nearby version that is *not* affected.

Done when: the CVE appears once (the advisory path de-dupes by CVE), and a neighbouring version produces no finding.

4. Teach the advisory database a new product

1. `advisories.py → PRODUCT_CPES`: add every (vendor, product) CPE pair the product appears under.
2. `advisories.py → product_key`: map the detected product name onto the key, with the usual ordering traps (mariadb before mysql, postgres before postgresql).
3. Make sure `scanner._extract_product` actually produces that product name from a banner or header.
4. `tests/test_advisories.py`: an ingestion case with a fake NVD document and a lookup case for a version in and out of range.

Done when: `--update-db` then a scan against a matching fixture yields a `VS-CVE-…` finding with NVD's own vector.

5. Add report content

1. Put the fact in `_coverage` (or `build_report`) first — every artifact renders from that dict.
2. Render it in `render_html`; if it belongs in the inventory, add the column to `CSV_COLUMNS` and the row in `write_csv`.
3. Bump `REPORT_SCHEMA_VERSION` if a new key appears, and note it in the README.
4. Add a `test_report_sections` case.

Done when: JSON, HTML, CSV and the PDF all show the new fact, and the schema version reflects the addition.

15 Failure modes and known limits

A tool is defined as much by how it fails as by what it finds. This section is the list you should be able to recite in a review, because every entry is a question a competent client will ask.

SYMPTOM	CAUSE, AND WHAT THE TOOL DOES ABOUT IT
Port open, no findings, no service	The listener accepted TCP then reset, went silent, or delays every reply (a tarpit). Recorded as `identification_error` and surfaced in coverage, the console and the CSV — never as a clean port.
Everything filtered	A firewall, IPS or rate limiter on the path. Ports are counted in `skipped_ports`; the report's limits section says absence of a finding is not proof of absence.
Service identified, version wrong	The banner lied or was rewritten by a proxy. Version matching is only as good as identification; no version, no CVE claim.
CVE matched on a patched host	Distro backport. Mitigated by detecting the package revision and downgrading to `tentative` with an explicit note — not eliminated, because the tool cannot read package changelogs.
CVE missed on a vulnerable host	The product is outside the ten advisory keys, the version is hidden, or the NVD entry has no usable range. The built-in fingerprints are the only fallback, and the report states which path was used.
Web app seen, findings are generic	One `GET /` is not a web assessment. Enumeration, authentication and business logic are explicitly out of scope (see `_NOT_TESTED`).
Host unreachable, scan "clean"	Targets are resolved once up front; an unresolvable name falls back to the literal string and produces no open ports. Check `hosts_up` / `hosts_down` in `metadata.scan` before believing a zero.
Different results from another network	Vantage point is part of the result. Egress filtering, geo-blocking and CDN edges change what is reachable. `coverage.scan_source` records where the scan ran.

SYMPTOM	CAUSE, AND WHAT THE TOOL DOES ABOUT IT
IPv6-only service invisible	The native engine resolves `AF_INET` only. A known limitation, not a policy: it is stated here so it is never a surprise in a report.
UDP services invisible	No UDP scan by design (DNS, NTP, SNMP, TFTP are common gaps). The limits section says so in every report.
Certificate details missing	DER decoding uses CPython's private `_test_decode_cert`. On failure the TLS checks fall back to handshake facts and the certificate rules simply do not fire.
nmap missing on the engagement box	`auto` falls back and says why; `--engine nmap` exits 2 rather than scanning with something else.

Three ways a finding can be wrong

1. **Wrong identification.** The port was named from a table rather than the wire. Mitigation: family gating, confidence labels, and the contradicted-port suppression in §4.
2. **Right identification, wrong version.** Banner rewriting, backports, or a patch level the banner does not mention. Mitigation: version ranges instead of exact matches, distro awareness, and tentative confidence.
3. **Right facts, wrong judgement.** The rule's condition does not actually mean what its severity implies. Mitigation: the rule matrix's negative case, the CVSS band invariant, and prose that states the condition plainly.

16 Study plan and self-test

Seven sittings, each with one file to read closely and questions you should be able to answer out loud before moving on. Commands are the ones that make the behaviour visible.

Sitting 1 — the shape of it (45 min)

Read `vulnscan/cli.py` end to end, then `vulnscan/targets.py`. Run `python3 -m vulnscan scanme.org -p 22,80 --no-pdf` and follow the console output against the code.

Then answer: what is the exit code when a scan finds nothing, and why is that the right choice for CI? Where is the report stem decided, and what happens with `-r "???"`? Which flag would you use to reuse a previous output folder without retyping it?

Sitting 2 — the data model (40 min)

Read the `PortService` dataclass and `report._service_dict`, then `report.build_report`.

Then answer: which four fields can be empty on an open service, and what does each emptiness mean? Why are `tls`, `http` and `scripts` omitted from the JSON when empty? Where would you add one new fact about a host so all four artifacts show it?

Sitting 3 — identity (60 min, the important one)

Read §4 above with `checks.py` lines 60–220 open: `SERVICE_FAMILIES`, `EXPECTED_FAMILIES`, `claims_conflict`, `_exposure_confidence`. Then read `tests/test_identity_rules.py`.

Then answer: why does a strong identity outside the expected set return `None` rather than a low-confidence finding? Why is `docker` not considered a conflict with `http`? What exactly makes an nmap claim strong, and why is `tcpwrapped` excluded even at `conf 8`?

Sitting 4 — engines and evidence (60 min)

Read `engines/nmap.py` and the identification half of `scanner.py` (`probe_port`, `classify_first_bytes`, `collect_evidence`). Run the same scan twice: `--engine nmap` and `--engine native`, and compare `coverage` in the two JSONs.

Then answer: how many TCP connections does a native probe of an SSH port use, and where is that asserted? What does `guess_names=False` change? Why is the raw nmap XML written beside the report instead of embedded in it?

Sitting 5 — judgement (60 min)

Read `checks.py` in rule order, then `cvss.py`, then `guidance.py`'s structure (not all 57 entries). Run `python3 -m unittest tests.test_cvss`.

Then answer: why are severity and CVSS band two separate fields, and what keeps them from disagreeing? Which rule would you expect to be `tentative` even with a perfect banner, and why? What does `entry_point` exclude on purpose?

Sitting 6 — the database (45 min)

Read `advisories.py` fully, then `tests/test_advisories.py`. If you want the real thing, run `python3 -m vulnscan --update-db --db-years 2024` into a scratch path and query it with `sqlite3`.

Then answer: what does a row with `exact_version` and no range mean? Why is `start_inclusive` stored separately from `introduced`? What happens to an NVD entry whose CPE says `version="*"` with no range? How does the report know the database's age?

Sitting 7 — prove it (60 min)

Read `tests/test_rule_matrix.py` and one rule's two cases. Then pick one rule and trace a single fixture from `PortService` to rendered HTML.

Then answer: what fails if you add a rule ID and no test? What is the negative case protecting against? Which command regenerates all four artifacts from one scan?

Self-test (answers follow)

---

1. A host has 3389 open and identified as SSH. Why is there no RDP finding?
The identity is strong and its family (`ssh`) is not in `EXPECTED_FAMILIES[3389]`, so `_exposure_confidence` returns `None` and the finding is never created. There is also no SSH exposure rule, so nothing is claimed in its place.

---

2. Is a `tentative` finding a warning about scanner uncertainty or about the host?
Both, precisely: it means something is listening on a port whose expected protocol could not be proved. It is still reported because suppression here would hide real exposure.

---

3. The scan says "no advisory database". Is that a bug or a configuration state?
A deliberate state. Scans never download anything; `--update-db` builds the file. The absence is stated in the console, in `metadata.advisory_db` and in the coverage table so a reader knows CVE matching was fingerprints-only.

4. Which engine's identification wins when both run?
nmap's, unless it is weak (`conf < 8`, `unknown`, trailing `?`). Then the evidence pass may rename the port from what the service actually sent.

5. Why is regreSSHion's finding `tentative` when the banner clearly matches?
Because distributions backport the fix without changing the upstream version. The banner proves the version, not the patch level, so the rule says so instead of overclaiming.

6. What tells a reader which detector actually ran?
`coverage.engine`, `engine_version`, `requested_engine`, `fallback_reason`, `argv` and `command` in the JSON, and the coverage table in the HTML.

7. Why is `-p all` no longer a false-positive factory?
Because a port only selects candidate rules; firing requires a protocol family match or an explicitly unidentified port, and the latter is labelled `tentative`.

8. Severity says high; the CVSS block says 5.3. What went wrong?
A rule bug: the curated vector does not match the declared band. `test_every_rule_vector_lands_in_its_rule_band` is designed to catch it in CI rather than in a report.

9. What does one connection per port buy you?
Fewer log lines and alerts on the target, less chance of tripping rate limits, and shorter scans. It is asserted, not assumed: `tests/test_sniffer.py` counts connections.

10. In the CSV, what is `identified_by` for, and which value should worry you?
It is `identity_source`. `port` means the name came from the port table, so the row is a guess; `banner`/`http`/`nmap` mean something proved it.

11. Can a rule ship without tests?
No. The matrix has a coverage check that fails when an ID has no positive or no negative case.

12. Why store `start_inclusive` when the column already has `introduced`?
NVD distinguishes `versionStartIncluding` from `versionStartExcluding`. The version string alone cannot express which one applied, so the flag is stored beside it.

13. What happens to an NVD entry with a bare wildcard version and no range?
It is skipped at ingestion: with no range and no fixed release it cannot be graded, and a guess would be indistinguishable from a fact.

14. List the seven things every report says it did not test.
UDP/SCTP and ports outside the list; authentication, credential guessing, exploitation and DoS; application business logic and anything behind a login; services filtered by firewall/IDS/rate limiting; TLS 1.0/1.1 unless `--legacy-tls`; vulnerabilities needing a version the banner does not reveal; NSE scripts outside the selected profile.

15. Why does `--strict` print a suppressed count?
Because silent suppression reads as a clean host. The count in `coverage.suppressed_by_strict` keeps the filter honest.

16. What happens with `--engine nmap` on a machine without nmap?
Exit 2 with an explicit message. It never quietly falls back — that is what `auto` is for.

17 Glossary

Banner	The opening bytes a service sends, read without being asked (server-first) or after one read-only probe.
Backport	A security fix applied to an older upstream version by a distributor. The version number does not change, which is why distro package revisions downgrade CVE findings to tentative.
CPE	Common Platform Enumeration, NVD's naming scheme for products (`cpe:2.3:a:apache:http_server:2.4.49:…`).
CIDR	Address range notation (`10.0.0.0/24`); targets larger than 1024 hosts are refused rather than truncated.
Confidence	How much was proved: `certain` (the service answered), `firm` (identity proven, consequence inferred), `tentative` (nothing proved the protocol).
Coverage	The report block listing what actually ran: engine, argv, privileges, scripts, skipped states, unidentified ports, advisory status, and what was not tested.
CVSS vector	A compact string of metric values (`AV:N/AC:L/…`) from which the base score is computed. The vector is stored; the score is derived.
Entry point	A finding a red team could use without first winning a credential — anonymous access or an unauthenticated management API.
Filtered / closed / open	Timed out or blocked / actively refused / accepted the connection. Only `open` is investigated; the others are counted.
First-bytes classification	Naming a protocol from what the service actually sent, before any port-table guess is allowed.
NSE	Nmap Scripting Engine. Only scripts nmap tags `safe` (plus read-only `intrusive` ones in `extended`) are ever run.
NVD	National Vulnerability Database; the source of the offline advisory database's CVE JSON 2.0 feeds.
Port-table guess	A service name taken from a port number rather than from the wire. Recorded as `identity_source: "port"` and treated as weak everywhere.
Provenance	Where a fact came from — engine, script, advisory build, guidance source, CVSS source — carried in the report so claims can be traced.
Service family	The normalised protocol group (`http`, `redis`, `docker`, …) that rules compare against. 31 families from 52 service names.
Tarpit	A listener that accepts connections and then stalls or resets, producing `tcpwrapped` in nmap and an identification gap here.
tcpwrapped	nmap's word for "the TCP handshake completed and then it said nothing". Not a service name, and never treated as one.
Weak identity	An identification that nothing proved: the port table, nmap below `conf 8`, an `unknown` or `tcpwrapped` name.